

System Call



Pengenalan Sistem Operasi

Mata Kuliah Sistem Operasi

OPERATING SYSTEM



Dosen Pengampu: Triawan Adi Cahyanto, M.Kom



Hardware



Program Studi: Teknik Informatika



Sistem Operasi



Universitas Muhammadiyah Jember



Aplikasi

RAM

I/O Devices

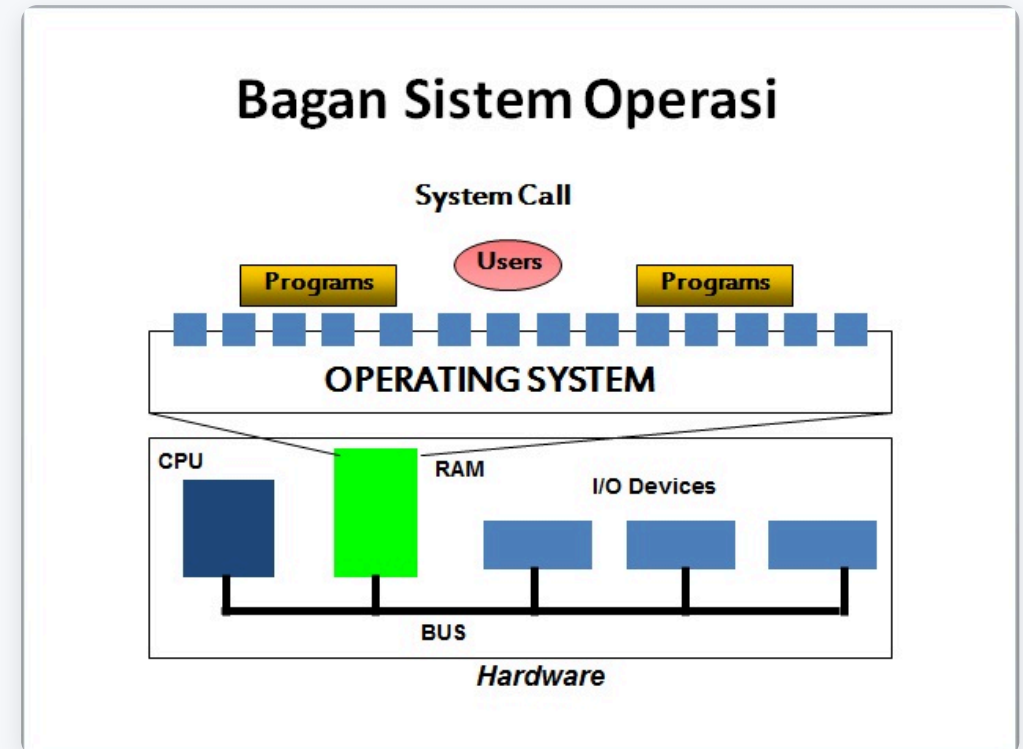
BUS

2025

Definisi Sistem Operasi

Sistem Operasi (OS) merupakan **perangkat lunak** yang bertindak sebagai **antarmuka** antara pengguna komputer dan perangkat keras komputer serta mengontrol pelaksanaan semua jenis program.

- ⚙ Mengelola sumber daya komputer (hardware & software)
- ↔ Menjembatani interaksi antara pengguna dan komputer
- 🔧 Mengontrol eksekusi program dan alokasi sumber daya
- 🛡 Menyediakan lingkungan yang aman dan efisien



Fungsi-fungsi Sistem Operasi

Sistem operasi memiliki berbagai fungsi penting untuk memastikan komputer berjalan dengan **efisien** dan **optimal**.



Manajemen Memori

Mengatur penggunaan memori utama, alokasi memori untuk proses, dan dealokasi memori saat tidak digunakan.



Manajemen Prosesor

Menjadwalkan penggunaan CPU, mengalokasikan prosesor ke proses, dan dealokasi saat proses selesai.



Manajemen Perangkat

Mengelola komunikasi perangkat melalui driver, mengalokasikan perangkat ke proses secara efisien.



Manajemen File

Mengorganisir file dalam direktori, mengatur hak akses, alokasi dan dealokasi sumber daya file.



Keamanan

Mencegah akses tidak sah ke program dan data melalui password dan teknik keamanan lainnya.



Kontrol Kinerja

Merekam penundaan antara permintaan layanan dan respons sistem untuk memantau performa.



Akuntansi Pekerjaan

Melacak waktu dan sumber daya yang digunakan oleh berbagai pekerjaan dan pengguna sistem.



Deteksi Kesalahan

Menghasilkan dump, jejak, pesan kesalahan, dan alat debugging untuk mendeteksi kesalahan.



Koordinasi

Mengkoordinasikan dan menugaskan compiler, interpreter, assembler, dan perangkat lunak lainnya.

Jenis-jenis Sistem Operasi

Berbagai jenis sistem operasi telah dikembangkan untuk memenuhi kebutuhan pengguna yang beragam.



Microsoft Windows

Sistem Operasi Desktop

Sistem operasi paling populer dengan pangsa pasar >80%. Antarmuka familiar dan banyak aplikasi pendukung.



Mac OS

Sistem Operasi Desktop

Dikembangkan oleh Apple untuk perangkat Macintosh. Dikenal dengan keamanan tinggi dan antarmuka intuitif.



Linux

Sistem Operasi Open Source

Sistem operasi open source yang fleksibel. Populer di kalangan pengembang dan administrator sistem.



Android

Sistem Operasi Mobile

Dikembangkan oleh Google untuk perangkat mobile. Fleksibel dan memiliki banyak aplikasi di Play Store.



iOS

Sistem Operasi Mobile

Dikembangkan oleh Apple untuk iPhone, iPad, dan iPod Touch. Dikenal dengan keamanan dan performa tinggi.



Cara Kerja Sistem Operasi

Sistem operasi menjalankan berbagai tugas penting untuk memastikan pengoperasian komputer yang **optimal** dan **efisien**.



Inisialisasi Sistem

Proses pertama saat komputer dinyalakan untuk mempersiapkan semua komponen sistem.

- ✓ Mengaktifkan **boot loader**
- ✓ Memeriksa **perangkat keras**
- ✓ Memuat **kernel** ke memori



Manajemen Memori

Mengelola penggunaan memori komputer agar dapat digunakan secara efisien oleh berbagai program.

- ✓ **Alokasi** memori untuk proses
- ✓ **Dealokasi** memori saat tidak digunakan
- ✓ Mengelola **memori virtual**



Penjadwalan Tugas

Mendistribusikan sumber daya komputer kepada berbagai tugas yang berjalan secara bersamaan.

- ✓ Menentukan **prioritas** proses
- ✓ Mengalokasikan **waktu CPU**
- ✓ Mengelola **multitasking**



Pengelolaan Sistem File

Mengorganisir dan mengelola file dan direktori dalam komputer.

- ✓ Mengatur **struktur direktori**
- ✓ Mengelola **hak akses** file
- ✓ Menyediakan **operasi I/O** file

Sejarah Perkembangan Sistem Operasi

1940-an

Era Manual

- Tidak ada sistem operasi
- Operasi dilakukan **manual**
- Operator mengatur langsung
- Penggunaan **toggle switch**

1950-an

Batch Processing

- Sistem operasi **pertama**
- Pemrosesan **batch**
- Input/output otomatis
- Monitor resident

1960-an

Multiprogramming & Timesharing

- **Multiprogramming**
- Sistem **timesharing**
- IBM System/360
- Multics & CTSS

1970-an

Personal Computer

- Sistem operasi **PC**
- UNIX dikomersialkan
- CP/M
- Apple DOS

1980-an

GUI

- **Graphical User Interface**
- MS-DOS & Windows 1.0
- Apple Macintosh
- OS/2

1990-an

Jaringan & Internet

- Sistem operasi **jaringan**
- Windows 95 & NT
- Linux **open source**
- Internet berkembang pesat

2000-an

Mobile & Cloud

- Sistem operasi **mobile**
- Windows XP & 7
- iOS & Android
- **Cloud computing**

2010-Sekarang

IoT & AI

- Sistem operasi untuk **IoT**
- Windows 10 & 11
- Integrasi **AI**
- Quantum computing

Struktur Sistem Operasi



Struktur Sederhana

Contoh: MS-DOS

Tidak terstruktur dengan baik, semua komponen saling terhubung tanpa batas yang jelas.

✓ Kelebihan

Sederhana
Kecil
Cepat

✗ Kekurangan

Sulit dimodifikasi
Tidak aman



Struktur Monolitik

Contoh: UNIX, Linux

Semua komponen sistem operasi berada di kernel yang sama dan berjalan di mode istimewa.

✓ Kelebihan

Performa tinggi
Komunikasi efisien

✗ Kekurangan

Besar & kompleks
Sulit dikembangkan



Struktur Berlapis

Contoh: THE OS

Sistem operasi dibagi menjadi beberapa lapisan, setiap lapisan hanya berinteraksi dengan lapisan di atas dan bawahnya.

✓ Kelebihan

Modular
Mudah dikembangkan

✗ Kekurangan

Kurang efisien
Komunikasi terbatas



Struktur Mikrokernel

Contoh: Windows NT, MINIX

Kernel minimal hanya berisi fungsi-fungsi esensial, layanan lainnya berjalan di ruang user.

✓ Kelebihan

Kecil & aman
Fleksibel

✗ Kekurangan

Performa lebih rendah
Komunikasi kompleks



Struktur Mesin Virtual

Contoh: IBM VM

Sistem operasi membuat ilusi beberapa mesin virtual, masing-masing dapat menjalankan OS sendiri.

✓ Kelebihan

Isolasi penuh
Multi OS

✗ Kekurangan

Overhead tinggi
Kompleks



Struktur Client-Server

Contoh: Mach, Windows NT

Sistem operasi terdiri dari proses server yang menyediakan layanan dan proses client yang meminta layanan.

✓ Kelebihan

Modular
Fleksibel

✗ Kekurangan

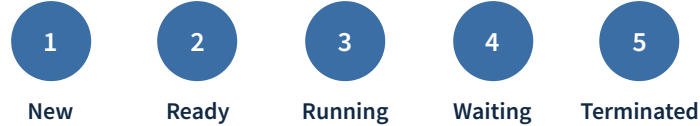
Komunikasi overhead
Performa bervariasi

Proses dan Thread

Definisi Proses

- ✓ Program dalam **eksekusi** dengan sumber daya sendiri
- ✓ Memiliki **konteks eksekusi** (PC, register, stack)
- ✓ Dikelola oleh sistem operasi melalui **scheduler**

State Proses



- i Proses berpindah antar state melalui **scheduler** dan **interrupt**

Process Control Block (PCB)

- i Struktur data yang menyimpan **informasi proses**

# Process ID	≡ Process State
⚙ Program Counter	🖨 CPU Registers
📁 Memory Limits	≡ List Open Files

Thread

- ✓ **Unit eksekusi** dalam proses
- ✓ Berbagi sumber daya proses (memori, file)

User-level Threads

Dikelola oleh **thread library** di ruang user

Kernel-level Threads

Dikelola langsung oleh **kernel OS**

Keuntungan Multithreading

- ⚡ **Responsivitas**
- 💰 **Ekonomis**
- 🔗 **Resource Sharing**
- 🖨 **Utilisasi Multiprosesor**

Isu Multithreading

- 🔑 **fork() dan exec()**
- 🔔 **Penanganan Sinyal**
- 🚫 **Pembatalan Thread**
- 🏊 **Thread Pools**

Penjadwalan CPU

Konsep Penjadwalan CPU

- ✓ Menentukan **proses mana** yang dieksekusi
- ✓ Berjalan ketika CPU **idle**
- ✓ Dikelola oleh **CPU scheduler**
- ✓ Memilih dari **ready queue**

Tujuan Penjadwalan

- ⚙️ **CPU Utilization**: Maksimalkan penggunaan CPU
- ↔️ **Throughput**: Jumlah proses per satuan waktu
- ⌚ **Turnaround Time**: Total waktu eksekusi
- 🕒 **Waiting Time**: Waktu menunggu di ready queue
- ⬅️ **Response Time**: Waktu dari request ke response

Preemptive vs Non-preemptive

Preemptive

Proses dapat **dihentikan** di tengah eksekusi

Lebih **responsif**

Memerlukan **overhead** context switching

Contoh: Round Robin, Priority

Non-preemptive

Proses berjalan hingga **selesai** atau menunggu I/O

Lebih **sederhana**

Overhead **lebih rendah**

Contoh: FCFS, SJF

Algoritma Penjadwalan

⬇️ **FCFS**: First-Come, First-Served

⬆️ **SJF**: Shortest-Job-First

⚠️ **Priority**: Berdasarkan prioritas

🔄 **RR**: Round Robin

📦 **Multilevel Queue**: Antrian bertingkat

🔄 **Multilevel Feedback**: Feedback queue

Evaluasi Algoritma

🕒 **Deterministic Modeling**

📊 **Queuing Models**

🔧 **Simulations**

🧪 **Implementation**

- 💡 Tidak ada algoritma **terbaik** untuk semua situasi
- ⚖️ Pilihan tergantung pada **kebutuhan sistem**

Faktor Pertimbangan

- 👤 **Jenis Sistem**: Batch, Interaktif, Real-time
- ⚙️ **Arsitektur CPU**: Single-core, Multi-core
- 📋 **Karakteristik Proses**: CPU-bound, I/O-bound
- ⚡ **Prioritas**: User, System, Real-time
- ⚖️ **Keadilan**: Fairness dalam alokasi CPU

Sinkronisasi dan Deadlock

Masalah Sinkronisasi

- ⚡ **Race Condition:** Output tergantung urutan eksekusi
- 🔒 **Critical Section:** Bagian kode mengakses shared data
- 🛡️ Solusi harus memenuhi: **Mutual Exclusion**, **Progress**, **Bounded Waiting**

Solusi Sinkronisasi

</> Peterson's Solution

🔧 Synchronization Hardware

🚦 Semaphores

🖥️ Monitors

Masalah Klasik

Bounded-Buffer
Produsen & Konsumen berbagi buffer terbatas

Readers-Writers
Pembaca & Penulis akses data bersama

Dining-Philosophers
5 filosof berbagi sumpit & makan

Deadlock

- 📄 **Definisi:** Set proses menunggu sumber daya yang dipegang proses lain

🚫 Mutual Exclusion

👉 Hold and Wait

⚡ No Preemption

🔄 Circular Wait

Penanganan Deadlock

Prevention

Mencegah salah satu kondisi necessary deadlock

Avoidance

Memerlukan informasi sumber daya sebelum alokasi

Detection & Recovery

Mendeteksi deadlock dan melakukan recovery

Algoritma Deadlock

Resource-Allocation Graph

Menggunakan graf untuk mendeteksi deadlock

Banker's Algorithm

Algoritma penghindaran deadlock untuk sumber daya tunggal

Manajemen Memori



Konsep Dasar

 **Alamat Logik:** Dilihat oleh CPU

 **Alamat Fisik:** Di memori unit

 **Binding:** Mapping alamat

 **Swapping:** Pindah proses



Fragmentasi

Internal

Memori terbuang di dalam partisi yang dialokasikan

Eksternal

Memori total cukup, tapi tidak kontigu



Contiguous Allocation

Fixed Partitioning

Partisi ukuran tetap, alokasi sederhana

Variable Partitioning

Partisi dinamis sesuai kebutuhan proses



Non-contiguous Allocation

Paging

Memori fisik dibagi menjadi frame

Segmentation

Memori logik dibagi menjadi segmen

Paged Segmentation

Kombinasi paging dan segmentation



Virtual Memory

 **Konsep:** Ilusi memori besar

 **Demand Paging:** Load on demand

FIFO

First In First Out

Optimal

Replace future page

LRU

Least Recently Used



Thrashing & Working Set

Thrashing

CPU lebih banyak melakukan paging daripada eksekusi

Working Set Model

Model untuk mencegah thrashing

Sistem File

Konsep Dasar File

Atribut File

Nama, jenis, lokasi, ukuran, perlindungan

Operasi File

Create, read, write, delete, truncate

Jenis File

Text, binary, executable, direktori

Struktur Direktori

Single-Level

Semua file di satu direktori

Two-Level

Direktori user & master

Tree-Structured

Hierarki direktori

Acyclic-Graph

Subdirektori dapat berbagi

General Graph

Memungkinkan siklus

Implementasi File System

 **Struktur Disk:** Partisi & cylinder

 **Boot Block:** Memulai sistem

 **Volume Block:** Info volume

 **Directory Structure:** File & direktori

 **Inode:** Metadata file

Metode Alokasi File

Contiguous

Block berurutan, akses cepat

Linked

Block terhubung pointer

Indexed

Index block ke semua block

Manajemen Ruang Bebas

 **Bit Vector:** Bit map block

 **Linked List:** Daftar block bebas

 **Grouping:** Kelompok block

 **Counting:** Jumlah block

Implementasi Direktori

Linear List

Daftar sederhana nama file & pointer

Hash Table

Hash function untuk pencarian cepat

Keamanan Sistem Operasi

Tujuan Keamanan

Confidentiality

Melindungi informasi dari akses tidak sah

Integrity

Menjaga keaslian dan konsistensi data

Availability

Memastikan sistem selalu dapat diakses

Ancaman Keamanan

 **Interruption:**
Mengganggu ketersediaan sumber daya

 **Interception:** Mencuri informasi

 **Modification:**
Mengubah data tanpa izin

 **Fabrication:** Membuat data palsu

Mekanisme Keamanan

 **Authentication:**
Verifikasi identitas

 **Authorization:** Kontrol akses

 **Encryption:** Enkripsi data

 **Auditing:** Pencatatan aktivitas

Proteksi Sistem

 **Domain Proteksi:**
Batasan akses

 **Access Matrix:** Tabel hak akses

 **Capability-Based:**
Token akses

 **Language-Based:**
Proteksi di level bahasa

Keamanan Jaringan

Firewall
Filter lalu lintas jaringan

IDS/IPS
Deteksi intrusi

VPN
Jaringan privat virtual

Keamanan Aplikasi

Buffer Overflow
Deteksi & pencegahan

Malware
Antivirus & antispysware

Secure Coding
Praktik pengembangan aman

Virtualisasi

Konsep Virtualisasi

- ✓ **Abstraksi** sumber daya fisik menjadi virtual
- ✓ Membuat **ilusi** beberapa sistem independen
- ✓ Memungkinkan **multiple OS** pada satu mesin
- ✓ Meningkatkan **utilisasi sumber daya**

Jenis Virtualisasi



Hardware: Virtualisasi perangkat keras



Software: Virtualisasi aplikasi



Memory: Abstraksi memori fisik



Storage: Virtualisasi penyimpanan



Network: Virtualisasi jaringan

Hypervisor

- i **Definisi:** Software yang membuat dan menjalankan VM

Type 1 (Bare Metal)

Berjalan langsung di hardware
Contoh: VMware ESXi, Xen

Type 2 (Hosted)

Berjalan di atas OS
Contoh: VirtualBox, VMware Workstation

Teknologi Virtualisasi

VMware

Solusi virtualisasi enterprise

VirtualBox

Open source, cross-platform

KVM

Kernel-based VM for Linux

Hyper-V

Microsoft virtualization

Xen

Type 1 hypervisor

Manfaat Virtualisasi



Konsolidasi Server



Isolasi



Portabilitas



Pengujian



Pengembangan

Tantangan Virtualisasi



Performa



Keamanan



Manajemen Sumber Daya

Sistem Operasi Terdistribusi



Konsep Sistem Terdistribusi

- ✓ **Kumpulan komputer otonom** yang terhubung jaringan
- ✓ Terlihat sebagai **satu sistem** oleh pengguna



Node 1



Node 2



Node 3



Tujuan Sistem Terdistribusi



Resource Sharing



Computation
Speedup



Reliability



Communication



Arsitektur Sistem Terdistribusi

Client-Server

Server menyediakan layanan untuk client

Peer-to-Peer

Setiap node dapat menjadi client dan server

Cluster

Kumpulan komputer yang bekerja sama

Grid

Sumber daya terdistribusi skala besar

Cloud

Layanan on-demand melalui internet



Isu-isu dalam Sistem Terdistribusi

Transparansi:



Menyembunyikan kompleksitas

Scalability:



Kemampuan untuk tumbuh

Fault Tolerance:



Ketahanan terhadap kesalahan



Concurrency: Kontrol akses bersamaan

Security:



Perlindungan data dan akses



Contoh Sistem Operasi Terdistribusi

Google Chrome OS

Berbasis cloud, ringan

Amoeba

Sistem mikrokernel

Plan 9

Dikembangkan Bell Labs

Sprite

Sistem file terdistribusi



Tren Terkini

Edge Computing

Proses data dekat sumber

Fog Computing

Ekstensi cloud computing

Distributed Ledger

Teknologi blockchain